

第4章 函数与方法

建议学时:4-6 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 `def` 定义、参数与返回值的完整语法,以及 `docstring` 规范
- **讲透"传对象引用"**:可变对象在函数内被修改会反映到外部,不可变对象不会;知道 `a = ...` 与 `a.append(...)` 的本质区别
- 掌握默认参数的经典坑:可变对象做默认值只初始化一次,正确姿势是 `None` 哨兵
- 掌握 `*args` / `**kwargs` 与关键字参数、`/` 与 `*` 分隔符(PEP 570/3102)
- 掌握多返回值(元组解构)、`lambda`、高阶函数(`map` / `filter` / `sorted` 的 `key`)
- 理解闭包与**晚期绑定**坑,知道循环变量捕获问题与修复
- 会用 `if __name__ == "__main__":` 组织可执行脚本

💡 从 C 到 Python:本章主题如何迁移

C 里函数是"函数",Python 里函数是**对象**——它可以被赋值给变量、塞进列表、作为参数传递、作为返回值,甚至可以给函数"附加属性"。C 用函数指针实现的回调,在 Python 里是日常语法;C 里"把函数传给函数"要小心翼翼写 `int (*cmp)(const void*, const void*)`,Python 直接写 `sorted(data, key=func)`。

最大的认知跨越在**参数传递**。C 是传值;想改外部变量,必须传指针。Python 是"传对象引用":参数拿到的是对象引用的**拷贝**——所以"重新给形参赋值"不影响外部(和传值一样),但"通过形参就地修改可变对象"会反映到外部(和传指针一样)。C 程序员需要同时丢弃两个直觉:①Python 没有"传引用给普通变量"这回事——函数内 `x = 新值` 只是让形参重新绑定;②Python 也不存在"值拷贝"——函数内 `list 参数.append(...)` 改的就是外面的那个对象。

第三个迁移点是**重载**。C 没有重载(除非 C11 `_Generic`);C++ 有重载。Python 没有重载:一个名字一个函数。替代方案是默认参数、`*args`、类型分派(`isinstance`)或干脆不同名字——实践中默认参数 + `None` 哨兵覆盖了 90% 的重载需求。

最后一个迁移点是闭包。C 的函数指针只能指向"静态的全局函数";Python 的函数可以**捕获创建它的作用域里的变量**,形成闭包。这让"工厂函数"成为可能:调用一次 `make_counter()` 得到一个带私有状态的函数。代价是晚期绑定坑:闭包捕获的是变量**本身**而非创建时的值——循环里生成的闭包全部共享同一个循环变量,本章会给出生产级修复。

📦 前置知识自查(你已会的 C 基础)

- C 函数的声明/定义/调用、参数与返回类型; `main(int argc, char *argv[])`
- 传值 vs 传指针: `void f(int x)` 与 `void f(int *p)` 的行为差异
- 函数指针与回调: `qsort` 的 `cmp` 参数
- 可变参数: `va_list` / `va_arg` (`printf` 的实现)
- `static` 函数与文件作用域; `errno` 错误返回约定

本章学习路线

1. **第一阶段(1.5 小时,语法):**读 §4.1–§4.3,重点做 §4.2 的"函数内修改是否影响外部"判断练习
2. **第二阶段(1.5 小时,参数与返回):**读 §4.4–§4.5,练 *args/**kwargs 与多返回值
3. **第三阶段(1.5 小时,一等函数与闭包):**读 §4.6–§4.8,复现晚期绑定坑并修复
4. **第四阶段(实验,1.5 小时):**完成章末实验"可配置统计管线"
5. **验收标准:**能不看资料回答"函数内 `lst.append(1)` 与 `lst = [1]` 对外部的影响分别是什么"

本章知识导图

- §4.1 函数定义与调用(def、docstring、类型标注)
- §4.2 参数传递:传对象引用(可变/不可变行为表)
- §4.3 默认参数与可变默认值经典坑
- §4.4 关键字参数、*args/**kwargs、参数分隔符
- §4.5 返回值:多返回值与 None 的语义
- §4.6 一等函数:lambda 与高阶函数
- §4.7 闭包与晚期绑定坑
- §4.8 递归与深度限制
- §4.9 入口与结构: `__name__ == "__main__"`

§4.1 函数定义与调用

语法要点: `def` 关键字、函数名、括号参数、冒号、缩进体。**没有返回类型声明**(返回值类型用标注表示,仅文档意义);没有函数原型/声明分离——定义即生效,但**调用必须发生在定义之后**(模块从上到下执行)。docstring(三引号字符串)是函数文档, `help(f)` 会显示它,生产代码必写。

示例 4-1 定义、docstring 与标注

```
def calc_avg(scores: list) -> float:
    """计算平均分。

    参数:
        scores: 数字列表(非空)
    返回:
        平均分;空列表返回 0.0
    """
    if not scores:
        return 0.0
    return sum(scores) / len(scores)

print(calc_avg([80, 90, 95]))      # 88.33333333333333
print(calc_avg([]))                # 0.0
# help(calc_avg)                  # 查看 docstring
# 注意:函数必须在调用前定义—模块按从上到下顺序执行
```

⚠ 误区 1:以为函数返回类型"强制"了返回值

-> float 只是标注,解释器不检查。返回 None 或字符串都"合法",直到运行时才可能炸。这与 C 的编译期类型检查完全不同——所以生产项目要么加 mypy(第14章),要么在函数开头用 assert /异常做防御性校验。标注的价值在文档与静态分析,不在运行时。

§4.2 参数传递:传对象引用(本章最重要)

精确语义只有一条:参数传递 = 把实参对象引用赋值给形参(等价于 param = arg)。由此推出全部行为:

函数内的操作	C 的等价	对外部的影响
x = 新对象 (给形参赋值)	传值: f(int x) 修改 x	无影响(形参重新绑定)
lst.append(1) (就地修改)	传指针: f(int *p) 修改 *p	有影响(对象被改)
s = s + "!" (str 拼接)	传值	无影响(不可变,产生新对象)
d["k"] = v (dict 写键)	传指针	有影响
n += 1 (int 运算)	传值	无影响(int 不可变)

示例 4-2 可变与不可变的行为对照

```
def demo(lst, n, s, d):
    lst.append(100)      # 就地修改 → 外部可见
    lst = [0, 0]         # 重新绑定 → 外部不可见(重要!)
    n = n + 1            # int 不可变 → 外部不可见
    s = s + "!"          # str 不可变 → 外部不可见
    d["new"] = 1         # dict 就地修改 → 外部可见

a, num, text, data = [1, 2], 10, "hi", {"k": 0}
demo(a, num, text, data)
print(a)                # [1, 2, 100]      append 生效
print(num, text)         # 10 hi          不受影响
print(data)              # {'k': 0, 'new': 1} 写键生效

# 一句话记忆:能就地改的(可变对象)就影响外部;
# 产生新对象的(不可变对象或重新赋值)就不影响外部
```

⚠ 误区 2:想"传引用"修改变量本身

C 程序员遇到"函数内改不了外部 int"时,第一反应是找引用语法。Python **没有**这种能力,也不该有:函数修改外部变量意味着隐藏耦合,是 bug 温床。正确姿势:①返回新值由调用方赋值;def f(x): return x + 1,然后 x = f(x);②把要改的东西装进可变容器(dict/list/对象属性);③或直接用一个类。生产代码里,函数要么返回结果,要么就地修改传入的可变对象,二选一,绝不混着来。

§4.3 默认参数与经典坑

默认参数语法 def f(a, b=10) 与 C++ 相同,但有一个 C++ 没有的坑:默认值在定义时求值一次,且被所有调用共享。用可变对象做默认值,等于让所有调用共用同一个列表——这是 Python 界排名前三的经典 bug。

示例 4-3 可变默认参数坑与修复

```
# 反例:全世界的 Python 初学者都会踩的坑
def add_item(item, bag=[]):          # 默认列表只创建一次!
    bag.append(item)
    return bag

print(add_item("苹果"))              # ['苹果']
print(add_item("香蕉"))              # ['苹果', '香蕉'] ← 上一次的残留!
# 不同调用者共享了同一个默认列表

# 正例:None 哨兵 + 函数内创建
def add_item_ok(item, bag=None):
    if bag is None:                  # 每次调用都新建列表
        bag = []
    bag.append(item)
    return bag

print(add_item_ok("苹果"))            # ['苹果']
print(add_item_ok("香蕉"))            # ['香蕉']

# 不可变默认值(数字/字符串/None)没有此问题,放心用
def greet(name, greeting="你好"):    # "你好" 是不可变 str
    return f"{greeting}, {name}"
```

⚠ 误区 3:以为默认参数“每次调用重新求值”

默认参数在 `def` 执行时求值一次(模块加载时),之后是同一个对象。可变的默认值(列表/字典/集合)被修改后,污染会保留给下一次调用。生产纪律:默认参数只允许不可变对象;需要可变就用 `None` 哨兵 + 函数内创建。这条规则比背一万个 API 都值钱。

§4.4 关键字参数、*args 与 **kwargs

Python 函数调用时可以用**关键字参数**按名字传参: `f(a=1, b=2)` ——调用顺序可以与定义顺序不同,这是 C 没有的能力。参数定义侧还可以用 `*args` (收位置参数的元组)与 `**kwargs` (收关键字参数的字典);调用侧用 `*iterable` 展开序列、`**dict` 展开映射。3.8+/3.10+ 的分隔符语法: `/` 前只能按位置传, `*` 后必须按关键字传。

示例 4-4 参数全家桶

```
def connect(host, port, timeout=30, *, use_ssl=True):
    """* 之后的参数必须按关键字传"""
    print(host, port, timeout, use_ssl)

connect("db.example.com", 5432)           # 位置 + 默认
connect("db.example.com", 5432, timeout=5) # 关键字覆盖默认
connect(host="h", port=1, use_ssl=False)   # 全关键字,顺序随意
# connect("h", 1, True)                   # TypeError: use_ssl 必须关键字

def log(level, *messages, **extra):
    """*messages 收任意多个位置参数,**extra 收任意关键字"""
    print(f"[{level}]", " ".join(str(m) for m in messages))
    for k, v in extra.items():
        print(f"    {k} = {v}")

log("INFO", "启动完成", "耗时 0.3s", user="alice", env="prod")

# 调用侧展开
params = {"host": "h", "port": 1}
connect(**params)           # 等价 connect(host="h", port=1)
nums = [1, 2, 3]
print(*nums)                # 1 2 3(等价 print(1, 2, 3))
```



工程建议:签名设计三原则

- ① 参数多于 4 个时,考虑把强相关的几个收进 `**kwargs` 或数据类(第14章);
- ② 布尔开关参数一律用关键字调用,调用处写 `send(data, retry=True)` 而不是 `send(data, 1, 0)`,否则没人知道 `True` 是谁;
- ③ `*args` / `**kwargs` 主要用于"转发"(写框架、装饰器,第12章),业务函数别滥用——丢失了签名可读性。

§4.5 返回值:多返回值与 None

C 的函数只能返回一个值,多值靠出参指针或结构体;Python 返回元组即可: `return a, b` (打包),调用方 `x, y = f()` (解包)。没有显式 `return` 的函数返回 `None`——它就是 C 的 `void` 加上"无值"标记。C 的错误返回约定(返回 `-1/errno`) 在 Python 里由**异常**取代(第9章);返回 `None` 表示"没有结果"是合法的数据语义。

示例 4-5 多返回值

```
def stats(nums: list) -> tuple:
    """返回 (总和, 个数, 平均值);空列表平均值为 None"""
    total = sum(nums)
    count = len(nums)
    avg = total / count if count else None
    return total, count, avg          # 打包成元组返回

total, count, avg = stats([10, 20, 30])  # 解包接收
print(total, count, avg)                # 60 3 20.0

# 丢弃不需要的值
_, n, _ = stats([1, 2, 3])
print(n)                                # 3

# C 的"出参"风格(不推荐):刻意不这么写
# def fill(buf): buf["x"] = 1  — 除非是就地修改已有容器
```

§4.6 一等函数:lambda 与高阶函数

函数是对象: `f = len`; `f([1,2])` 合法;函数可以放进 list/dict,可以传参、返回。lambda 是单表达式匿名函数,等价于 C 的回调函数指针——但写法短得多。对照关系:

C 的做法	Python 的做法
<code>qsort(a, n, sz, cmp)</code> 传比较函数指针	<code>sorted(a, key=func)</code> 传取键函数
<code>void (*cb)(int);</code> <code>register_cb(cb);</code>	<code>cb = lambda x: print(x); register_cb(cb)</code>
函数只能全局定义	函数可嵌套、可捕获变量(闭包)
回调里传状态要 <code>void* ctx</code>	闭包直接捕获,无需额外参数

Python 官方推荐的高阶函数风格:

- `sorted(data, key=func)` — 按 key 排序,取代 C 的 `qsort` + 比较回调
- `map(func, it)` / `filter(func, it)` — 批量映射/过滤(常被列表推导替代,但 map 惰性)
- `functools.reduce` — 归约(求积等;可读性差,慎用)
- 把函数存进 dict 做"分发表"(第3章实验已见)

示例 4-6 高阶函数与 lambda

```

users = [
    {"name": "李雷", "age": 18},
    {"name": "韩梅梅", "age": 17},
    {"name": "张三", "age": 22},
]

# key 参数:排序的"取键函数"(取代 qsort 的比较回调)
by_age = sorted(users, key=lambda u: u["age"])
print([u["name"] for u in by_age])      # 韩梅梅 李雷 张三

by_name = sorted(users, key=lambda u: u["name"])  # 字符串排序

# map/filter:惰性迭代器
ages = map(lambda u: u["age"], users)      # 不立即执行
print(list(ages))                          # [18, 17, 22]

adults = filter(lambda u: u["age"] >= 18, users)
print([u["name"] for u in adults])        # 李雷 张三

# 函数作返回值:工厂
def make_multiplier(k):
    def mul(x):
        return x * k
    return mul

double = make_multiplier(2)
print(double(21))                        # 42

```

工程建议:lambda 用不用

lambda 体只能有一个表达式——逻辑超过一行,立刻改用 `def` 并起个名字。生产判据:给这个 lambda 起名后,名字能否说明意图?能(`key=lambda u: u["age"]`),用;不能,写 `def`。另外排序的 `key` 函数调用次数是 $O(n \log n)$, 昂贵操作(如解析时间戳)请先缓存(`users.sort(key=cache)` 或提前构造键值对)。

§4.7 闭包与晚期绑定坑

闭包 = 函数 + 它捕获的外部变量。上面的 `make_multiplier` 就是闭包: `mul` 捕获了 `k`。C 的函数指针做不到这点——被指向的函数没有“记忆”。但闭包有一个著名陷阱:**晚期绑定**——闭包捕获的是变量本身,不是创建时的值。循环里创建的一批闭包,全部共享循环结束后的最终值:

示例 4-7 晚期绑定坑与修复

```
# 坑:循环里生成 3 个闭包,记住 i=0,1,2
funcs = []
for i in range(3):
    funcs.append(lambda: i)          # 捕获的是变量 i 本身!

print([f() for f in funcs])          # [2, 2, 2] ← 全是 2!

# 修复一:默认参数立即绑定值(推荐,简单)
funcs = []
for i in range(3):
    funcs.append(lambda i=i: i)      # i=i:默认参数在定义时求值
print([f() for f in funcs])          # [0, 1, 2]

# 修复二:工厂函数(更明确)
def make_fn(i):
    return lambda: i

funcs = [make_fn(i) for i in range(3)]
print([f() for f in funcs])          # [0, 1, 2]
```

⚠ 误区 4:以为闭包"拍快照"了创建时的值

闭包捕获的是**变量单元**而非值。在循环/回调场景(定时器、事件、线程池提交任务)里,所有闭包会看到循环变量的最终值。修复首选"默认参数绑定" `lambda i=i: i`;语义上它把当前值固化到函数自己的局部变量里。生产事故经典场景:for 循环里给按钮绑定回调,点哪个都是最后一个——记住这个坑,第9章线程池还会再见它。

§4.8 递归与深度限制

Python 支持递归,但有**深度限制**(默认 1000, `sys.getrecursionlimit()` 可查)。C 的递归受栈大小限制(通常几万层);Python 的每个调用帧开销大,1000 层就会抛 `RecursionError`。没有尾递归优化(PEP 622 之类一直未落地),不要指望编译器帮你。生产结论:深度可能超过几百的递归(树/图遍历通常没问题,链表求长等线性递归要小心),改写成迭代或用显式栈。

示例 4-8 递归与迭代改写


```

import sys
print(sys.getrecursionlimit())      # 1000(默认)

# 经典递归:目录树节点求和(深度可控,放心用)
def tree_sum(node):
    total = node.get("value", 0)
    for child in node.get("children", []):
        total += tree_sum(child)
    return total

# 线性递归:深度 = n,大 n 会炸
def fact(n):
    return 1 if n <= 1 else n * fact(n - 1)

# 改迭代:显式栈(深度任意)
def fact_iter(n):
    result = 1
    while n > 1:
        result *= n
        n -= 1
    return result

print(fact(10), fact_iter(10))      # 3628800 3628800
# sys.setrecursionlimit(10000)      # 可调,但只是推迟问题

```

§4.9 入口与结构: `__name__ == "__main__"`

C 程序的入口是 `main()`; Python 没有固定入口——模块顶层代码直接执行。习惯上把"运行时动作"放进 `main()`, 并用 `if __name__ == "__main__":` 保护:

C 程序	Python 脚本
<code>int main(int argc, char **argv)</code>	<code>def main(): ...</code> (无强制签名)
运行即从 <code>main</code> 开始	顶层代码从上到下执行
—(无"导入"概念)	<code>if __name__ == "__main__": main()</code> 防导入副作用
<code>argc / argv</code>	<code>sys.argv</code> (第7章)

直接运行本文件时 `__name__` 是 `"__main__"`, 被 `import` 时是模块名——于是同一个文件既能当脚本跑, 又能被导入复用函数而不会误触发副作用。这是 Python 脚本组织的第一纪律。

示例 4-9 `main` 守卫

```
# tools.py
def fetch_data(url: str) -> str:
    return f"模拟数据:{url}"

def process(text: str) -> int:
    return len(text)

def main() -> None:
    data = fetch_data("https://example.com")
    print(f"处理结果:{process(data)}")

if __name__ == "__main__":
    main()          # python tools.py 时执行
# 被 import tools 时,main() 不会执行—这是关键
```

本章速查表

主题	写法 / 结论
定义	<code>def f(a, b=10) -> int:</code> 冒号 + 缩进;docstring 三引号
调用	位置参数、关键字参数 <code>f(a=1)</code> 、混用(位置在前)
传参语义	传对象引用:就地修改可变对象影响外部;重新赋值不影响
改外部变量	没有引用语法;用返回值或就地修改可变对象
默认参数	定义时求值一次;可变默认值用 <code>None</code> 哨兵
<code>*args</code>	收位置参数为元组; <code>print(*nums)</code> 调用侧展开
<code>**kwargs</code>	收关键字参数为字典; <code>connect(**params)</code> 展开
分隔符	<code>/</code> 前只能位置; <code>*</code> 后必须关键字(3.8+/3.10+)
多返回值	<code>return a, b + x, y = f(); _</code> 丢弃
无返回	返回 <code>None</code> ,等价 C 的 <code>void</code>
lambda	单表达式; <code>sorted(data, key=lambda u: u["age"])</code>
map/filter	惰性迭代器;多与推导式二选一
闭包	函数捕获外层变量;循环闭包有晚期绑定坑
晚期绑定修复	<code>lambda i:i: i</code> 或工厂函数
递归	默认上限 1000;线性递归改迭代或显式栈
入口	<code>if __name__ == "__main__": main()</code>

最小必会清单

- 能说出"传对象引用"的精确语义,并判断任意一段"函数内修改代码"是否影响外部

- 能解释可变默认参数 bug 的机制,并写出 None 哨兵修复版
- 能写出 *args/**kwargs 的定义与调用侧展开
- 能解释 / 与 * 分隔符的作用(PEP 570/3102)
- 能写出多返回值函数与解包调用
- 能用 key 参数实现按字典字段排序
- 能解释 lambda 捕获循环变量的坑,并给出两种修复
- 能写出闭包计数器/乘法器工厂
- 能说出递归上限默认值与 RecursionError 的应对
- 能写出 __name__ 守卫并解释 import 与直接运行的区别

自测题

Q 1. 写出下列代码的最终输出: `def f(a, lst=[]): lst.append(a); return lst`,依次调用 `f(1)`、`f(2)`、`f(3, [9])`。

✓ 参考答案

输出 `[1]`、`[1, 2]`、`[9, 3]`。前两次共享同一个默认列表(定义时创建),第二次看到第一次的残留;第三次显式传了新列表。修复: `def f(a, lst=None): lst = [] if lst is None else lst`。

Q 2. 函数 `def g(x): x.append(1); x = [9]; return x`,调用 `a = [0]; r = g(a)` 后, `a` 与 `r` 分别是什么?

✓ 参考答案

`a == [0, 1]` (append 就地修改,外部可见); `r == [9]` (重新绑定只影响形参)。这题是传参语义的照妖镜:append 改对象,赋值换标签。

Q 3. 用 lambda + sorted 按字符串长度从短到长排序 `words = ["python", "c", "rust", "java"]`。

✓ 参考答案

`sorted(words, key=lambda w: len(w))` → `['c', 'rust', 'java', 'python']`。要点:key 是"取键函数",不是比较函数——这是与 `qsort` 比较回调的本质区别。

Q 4. 为什么下面的代码输出 `[2, 2, 2]`?给出修复。 `funcs = [lambda: i for i in range(3)]; print([f() for f in funcs])`

✓ 参考答案

晚期绑定:lambda 捕获的是循环变量 `i` 本身,调用时 `i` 已是最终值 2。修复: `[lambda i=i: i for i in range(3)]` 或 `[make(i) for i in range(3)]` 其中 `def make(i): return lambda: i`。

Q 5. 用 *args 写一个函数 `join_all(sep, *parts)`,把 `parts` 用 `sep` 连接后返回。

✅ 参考答案

```
def join_all(sep, *parts):
    """用 sep 连接任意多个 parts; sep 是普通位置参数"""
    return sep.join(parts)

print(join_all("-", "a", "b", "c"))    # a-b-c
print(join_all(" ", "Python", "3.10")) # Python 3.10
print(join_all("+", *[1, 2, 3]))        # 1+2+3(调用侧展开)
```

Q 6. 一个函数既有可变默认参数又有递归,深度 2000 的递归会怎样?如何验证与应对?

✅ 参考答案

抛 `RecursionError: maximum recursion depth exceeded` (默认上限 1000)。验证: `sys.getrecursionlimit()`; 应对: 改写为迭代/显式栈, 或万不得已 `sys.setrecursionlimit(5000)` (只是推迟, 且可能触发 C 栈溢出崩溃, 生产慎用)。

章末实验题: 可配置统计管线

📁 实验 4: 可配置的数组统计/过滤管线

实验目标: 用函数式风格(回调、闭包、高阶函数、多返回值全覆盖)实现一个数据统计管线: 一段数字经过“过滤 → 变换 → 统计”三步, 每一步都可配置。

实验要求:

- 任务 1: 写 `filter_data(data, keep)`, `keep` 是谓词回调(如“只保留正数”), 返回新列表且不修改原列表(知识点: 传对象引用、回调、列表推导)
- 任务 2: 写 `map_data(data, transform)`, `transform` 是变换回调(如平方、放大倍数)(知识点: 高阶函数)
- 任务 3: 写 `make_range_checker(lo, hi)` 闭包工厂, 返回“判断 `x` 是否在 `[lo, hi]`”的谓词; 验证不同工厂实例互不干扰(知识点: 闭包、工厂)
- 任务 4: 写 `make_scaler(k)` 闭包工厂, 返回乘以 `k` 的变换函数; 同时演示晚期绑定坑, 用 `lambda k=k`: 修复后给三个缩放器 `k=1, 2, 3` 分别映射(知识点: 晚期绑定与修复)
- 任务 5: 写 `statistics(data)` 返回 (总和, 个数, 平均值, 最大, 最小) 五元组, 空列表返回 `None` 平均(知识点: 多返回值)
- 任务 6: 主流程用 `if __name__ == "__main__":` 驱动: 过滤 → 变换 → 统计 三步, 每步打印输入/输出; 默认参数全部用 `None` 哨兵风格设计(知识点: `main` 守卫、默认参数)

实验环境: Python 3.10+, 单文件 `pipeline.py`。

第一步 写三个工厂函数(`make_range_checker`/`make_scaler`), 它们返回闭包——先跑通“闭包有记忆”的直觉

第二步 `filter_data` 与 `map_data` 内部用列表推导实现, 保持“不修改入参”的纯函数风格

第三步 统计函数返回五元组, 调用处用 `total, count, avg, mx, mn = ...` 解包

第四步 后用守卫把三阶段串起来, 每个阶段打印前后数据, 肉眼验证

✅ 参考答案要点(代码分两段)

第一段:纯函数与闭包工厂。核心设计:数据流单向(入参只读、返回新列表),工厂闭包携带私有状态——这是函数式风格的骨架。

```
# pipeline.py — 可配置统计管线(覆盖第4章全部知识点)

def filter_data(data: list, keep) -> list:
    """按谓词 keep 过滤,返回新列表,不修改入参"""
    return [x for x in data if keep(x)]

def map_data(data: list, transform) -> list:
    """按变换 transform 映射,返回新列表"""
    return [transform(x) for x in data]

def make_range_checker(lo, hi):
    """闭包工厂:返回"判断 x 是否在 [lo, hi]"的谓词"""
    def keep(x):
        return lo <= x <= hi      # 链式比较
    return keep

def make_scaler(k):
    """闭包工厂:返回乘以 k 的变换函数"""
    def scale(x):
        return x * k
    return scale

def statistics(data: list) -> tuple:
    """多返回值:返回 (总和, 个数, 平均值, 最大, 最小)"""
    total = sum(data)
    count = len(data)
    avg = total / count if count else None
    return total, count, avg, max(data, default=None), min(data, default=None)
```

第二段:主流程。演示晚期绑定坑与修复——循环里生成三个缩放器,不修复则全部按最后一个 k 缩放。

```

def main() -> None:
    raw = [3, -1, 7, 0, 4, -5, 9, 2]

    # 1. 过滤:闭包工厂产生谓词
    keep_positive = make_range_checker(0, 10**9)
    step1 = filter_data(raw, keep_positive)
    print(f"过滤后: {step1}")

    # 2. 变换:演示晚期绑定坑
    # 反例:三个缩放器共享循环变量 k,全变成 x3
    bad = [make_scaler(k) for k in (1, 2, 3)]
    print(f"坑: {[f(10) for f in bad]}")          # [30, 30, 30]

    # 修复:默认参数立即绑定
    def make_scaler_fixed(k, _k=k):
        return lambda x: x * _k

    ok = [make_scaler_fixed(k) for k in (1, 2, 3)]
    print(f"修复: {[f(10) for f in ok]}")        # [10, 20, 30]

    # 3. 变换:平方后取正数区
    step2 = map_data(step1, lambda x: x * x)
    step3 = filter_data(step2, keep_positive)
    print(f"平方后: {step2}")

    # 4. 统计:多返回值解包
    total, count, avg, mx, mn = statistics(step3)
    print(f"个数={count} 总和={total} 平均={avg} "
          f"最大={mx} 最小={mn}")

    # 5. 空列表语义:平均值为 None
    print(statistics([]))                      # (0, 0, None, None, None)

if __name__ == "__main__":
    main()

```

设计说明:① 管线各阶段是纯函数(只读入参、返回新列表),可以任意组合、单独测试——这是函数式风格的工程收益;② 晚期绑定修复用了"默认参数立即绑定"技巧,并在实验里同时演示"坑"与"修复",读代码的人一眼看懂两种行为;③ 统计函数返回五元组,主流程一次性解包,避免了 5 个全局变量;④ main 守卫保证本文件既可 `python pipeline.py` 运行,也可被导入复用 `filter_data` 等函数。

下一章预告:第5章 内存管理——引用计数与分代 GC 的工作原理、gc 模块、循环引用与 `__del__` 的坑、with 上下文管理器,以及"在 GC 语言里内存泄漏怎么发生"。